Seguridad y Calidad en el Desarrollo (CDY2203)

Informe escrito

Ignacio Kriman Bravo

04/05/2026

Índice

Tabla comparativa…….3

Herramienta seleccionada y justificación……..4

Proceso de implementación……4

Ejecucion JaCoCo……....5

Analisis JaCoCo……...5

Resultados obtenidos…….5

Evidencias……6

Tabla comparativa

Herramienta	*Tipo*	*Organización responsable*	*Características principales*	*URL*
JaCoCo	Software libre (EPL 2.0)	Proyecto JaCoCo / comunidad open source	Alta integración con Maven, Gradle y Jenkins. Genera reportes detallados en HTML, XML y CSV. Mide cobertura de instrucciones, líneas, ramas, métodos y clases. Es liviana, rápida y ampliamente utilizada en proyectos Java modernos.	https://www.jacoco.org/jacoco/
Cobertura	Software libre	Comunidad open source	Herramienta histórica para Java. Permite medir cobertura de líneas y ramas. Se integra con Maven y Ant. Aunque sigue siendo funcional, su desarrollo es menos activo que otras alternativas.	https://cobertura.github.io/cobertura/
OpenClover	Software libre (desde 2017)	Originalmente Atlassian, actualmente comunidad open source	Ofrece métricas avanzadas, análisis de cobertura por prueba y reportes visuales muy detallados. Compatible con Java, Groovy y otros lenguajes de la JVM. Ideal para proyectos grandes y análisis profundos.	https://openclover.org/

Herramienta seleccionada y justificación

Para medir la cobertura de pruebas del sistema, se seleccionó la herramienta **JaCoCo (Java Code Coverage)**, una de las soluciones más utilizadas en proyectos desarrollados en Java. Esta herramienta permite analizar de manera precisa qué porcentaje del código fuente es ejecutado durante la ejecución de las pruebas unitarias.

La elección de JaCoCo se fundamenta en varias razones. En primer lugar, se integra de forma nativa con herramientas de construcción como Maven, lo que facilita su incorporación al ciclo de desarrollo sin requerir configuraciones complejas. Además, genera reportes detallados y visuales sobre cobertura de instrucciones, ramas, líneas, métodos y clases, permitiendo identificar con claridad las áreas del sistema que requieren mayor atención.

Otra ventaja relevante es que JaCoCo se complementa perfectamente con frameworks de pruebas como JUnit 5 y Mockito, utilizados en este proyecto para la creación y ejecución de pruebas automatizadas. Esto permitió evaluar tanto la cobertura general del sistema como el impacto específico de las pruebas implementadas en los componentes críticos de la aplicación.

Finalmente, la utilización de JaCoCo contribuye directamente al aseguramiento de la calidad del software, ya que facilita la detección de código no probado, reduce riesgos de fallos en producción y promueve una estrategia de pruebas más robusta y mantenible.

Proceso de implementación

La implementación de JaCoCo se realizó integrándolo al ciclo de construcción del proyecto mediante Maven. Para ello, se configuró el plugin correspondiente en el archivo pom.xml, permitiendo que el análisis de cobertura se ejecutara automáticamente junto con las pruebas unitarias.

Posteriormente, se desarrollaron y ejecutaron pruebas unitarias utilizando JUnit 5 y Mockito, enfocándose en los componentes críticos del sistema, especialmente controladores y servicios. Esta estrategia permitió validar tanto escenarios exitosos como situaciones excepcionales, incluyendo autenticación, manejo de sesiones y operaciones sobre recursos principales.

Una vez ejecutadas las pruebas, se generó el reporte de cobertura mediante el comando mvn clean test jacoco:report. El informe resultante permitió identificar las clases con menor cobertura, facilitando la priorización de nuevas pruebas en aquellas áreas con mayor impacto en la calidad y confiabilidad del sistema.

Ejecucion JaCoCo

Tras la ejecución de JaCoCo, se obtuvo una cobertura total de instrucciones del 57% y una cobertura de ramas del 36%. Estos resultados evidencian que una parte importante del código ya se encuentra validada mediante pruebas unitarias. Sin embargo, la cobertura total aún se encuentra por debajo del umbral mínimo exigido del 60%, por lo que fue necesario desarrollar pruebas adicionales para aumentar el nivel de cobertura y cumplir con el estándar establecido.

Analisis JaCoCo

El análisis del reporte de JaCoCo permitió identificar las clases con menor nivel de cobertura, destacando **PetRestController** (6%), **AuthController** (13%), **PatientRestController** (14%) y **AppointmentRestController** (14%). Asimismo, algunas clases del modelo, como **InvoiceAppointment** e **InvoiceLineItem**, presentaron una cobertura del 0%.
Debido a su bajo porcentaje y alto impacto en la cobertura global, se priorizó el desarrollo de nuevas pruebas unitarias para los controladores REST, particularmente **PetRestController** y **AuthController**. Esta estrategia permitió aumentar de manera eficiente la cobertura total del sistema, enfocando los esfuerzos en los componentes más críticos y menos evaluados.

Resultados obtenidos

Antes de la incorporación de nuevas pruebas, el proyecto presentaba una cobertura de instrucciones cercana al 57%, valor insuficiente para cumplir con el requisito mínimo establecido. Tras la implementación de nuevas pruebas unitarias en controladores y servicios clave, la cobertura aumentó significativamente.

Como resultado final, el sistema alcanzó una cobertura de 67% de instrucciones y 59% de ramas, superando ampliamente el mínimo exigido de 60%. Además, se ejecutaron exitosamente 89 pruebas automatizadas, sin registrar fallos ni errores.

Este incremento evidencia una mejora sustancial en la validación del comportamiento del sistema, reduciendo el riesgo de defectos y aumentando la confianza en futuras modificaciones del código.

Evidencias

pom.xml

C: > Users > ignac > Desktop > DUOC > 01 - RAMOS > 012 - Seguridad y Calidad en el Desarrollo > s8 > cdy2203-2026-201-main > c

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema
    <build>
        <plugins>
            <groupId>org.jacoco</groupId>
            <artifactId>jacoco-maven-plugin</artifactId>
            <version>0.8.12</version>
            <executions>
                <execution>
                    <goals><goal>prepare-agent</goal></goals>
                </execution>
                <execution>
                    <id>report</id>
                    <phase>test</phase>
                    <goals><goal>report</goal></goals>
                </execution>
            </executions>
        </plugin>
                <plugin>
                    <groupId>org.jacoco</groupId>
                    <artifactId>jacoco-maven-plugin</artifactId>
                    <version>0.8.14</version>
                    <executions>
                        <execution>
                            <id>default-prepare-agent</id>
                            <goals>
                                <goal>prepare-agent</goal>
                            </goals>
                        </execution>
                        <execution>
                            <id>default-report</id>
                            <goals>
                                <goal>report</goal>
                            </goals>
                        </execution>
                        <execution>
                            <id>default-check</id>
                            <goals>
                                <goal>check</goal>
                            </goals>
                            <configuration>
                                <rules>
                                    <rule>
                                        <element>BUNDLE</element>
                                        <limits>
                                            <limit>
                                                <counter>LINE</counter>
                                                <value>COVEREDRATIO</value>
                                                <minimum>0.60</minimum>
                                            </limit>
```

Incorporación del plugin de JaCoCo en el POM del proyecto.

```
Windows PowerShell
s   : Starting SeguridadcalidadApplicationTests using Java 21.0.6 with PID 19156 (started by ignac in C:\Users\ignac\Des
ktop\DUOC\01 - RAMOS\012 - Seguridad y Calidad en el Desarrollo\s8\cdy2203-2026-201-main\cdy2203-2026-201-main)
2026-05-01T19:07:36.368-04:00  INFO 19156 --- [seguridadcalidad] [           main] c.d.s.SeguridadcalidadApplicationTest
s   : No active profile set, falling back to 1 default profile: "default"
2026-05-01T19:07:36.659-04:00  INFO 19156 --- [seguridadcalidad] [           main] r$InitializeUserDetailsManagerConfigu
rer : Global AuthenticationManager configured with UserDetailsService bean with name users
2026-05-01T19:07:36.727-04:00  INFO 19156 --- [seguridadcalidad] [           main] c.d.s.SeguridadcalidadApplicationTest
s   : Started SeguridadcalidadApplicationTests in 0.387 seconds (process running for 7.17)
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.399 s -- in com.duoc.seguridadcalidad.Seguridad
calidadApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 66, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.14:report (report) @ seguridadcalidad ---
[INFO] Loading execution data file C:\Users\ignac\Desktop\DUOC\01 - RAMOS\012 - Seguridad y Calidad en el Desarrollo\s8\
cdy2203-2026-201-main\cdy2203-2026-201-main\target\jacoco.exec
[INFO] Analyzed bundle 'seguridadcalidad' with 27 classes
[INFO]
[INFO] --- jacoco:0.8.14:report (default-cli) @ seguridadcalidad ---
[INFO] Loading execution data file C:\Users\ignac\Desktop\DUOC\01 - RAMOS\012 - Seguridad y Calidad en el Desarrollo\s8\
cdy2203-2026-201-main\cdy2203-2026-201-main\target\jacoco.exec
[INFO] Analyzed bundle 'seguridadcalidad' with 27 classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  12.460 s
[INFO] Finished at: 2026-05-01T19:07:37-04:00
```

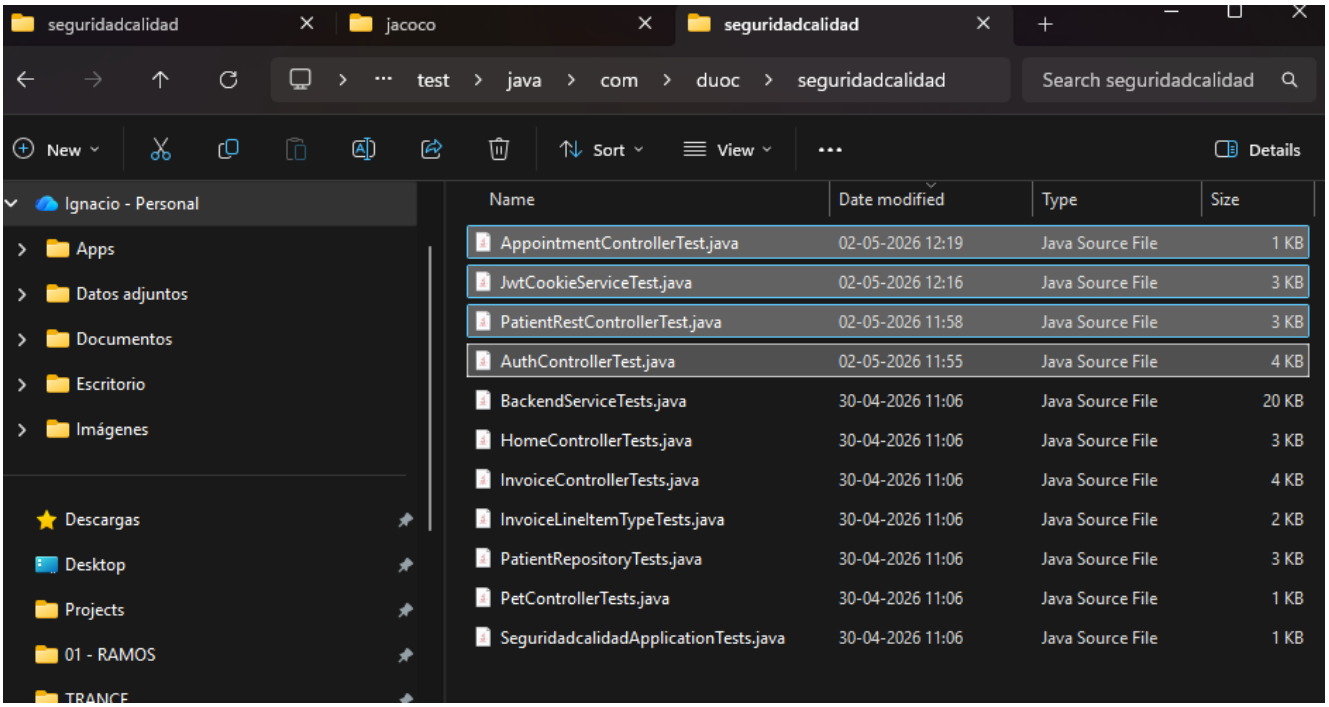
Ejecución del comando en la terminal

seguridadcalidad

seguridadcalidad

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.duoc.seguridadcalidad		57%		36%	124	205	278	572	99	169	4	27
Total	943 of 2,224	57%	46 of 72	36%	124	205	278	572	99	169	4	27

Cobertura previa a las mejoras con un 57%



Incorporación de varias pruebas.

seguridadcalidad

seguridadcalidad

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.duoc.seguridadcalidad		67%		59%	106	205	224	572	89	169	4	27
Total	722 of 2,224	67%	29 of 72	59%	106	205	224	572	89	169	4	27

Resultado post pruebas y ajustes del codigo subiendo a un 67%